# Lab 5: Static Code Analysis

**Submission Report** | Student: **Sagar Walikar** | SRN: **PES1UG24CS396**

| | |
|---|---|
| **Assigned Script:** expense_tracker.py | **Tools Used:** Flake8, Bandit, Pylint |
| **Initial Pylint Score:** 5.27 / 10.00 | **Final Pylint Score:** 10.00 / 10.00 |
| **Security Findings:** 4 resolved (0 remaining) | **Git Commits:** 3 separate tool-specific commits |

## 1. Introduction to Static Code Analysis and the Tools

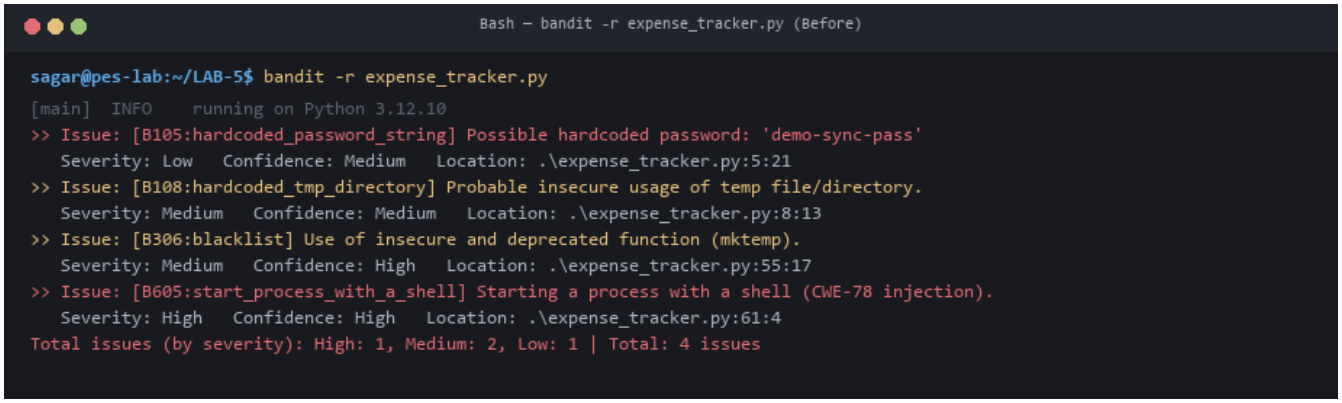
Static code analysis is the process of examining source code without running it to find syntax errors, formatting problems, code quality issues, and security vulnerabilities early in development.

In this lab, three tools were used to analyze and fix expense_tracker.py:
• **Flake8:** A tool that checks code against PEP 8 style guidelines (indentation, blank lines, whitespace, line length) and checks for basic syntax mistakes or unused variables.
• **Bandit:** A security linter built specifically for Python. It parses the Abstract Syntax Tree (AST) to find common security vulnerabilities such as hardcoded passwords, shell injections, and unsafe temporary file usage.
• **Pylint:** A code quality reviewer that looks at programming standards, code design, docstring completeness, naming conventions (snake_case), and bad programming patterns (like catching broad exceptions or using global variables). It provides a score out of 10.0.

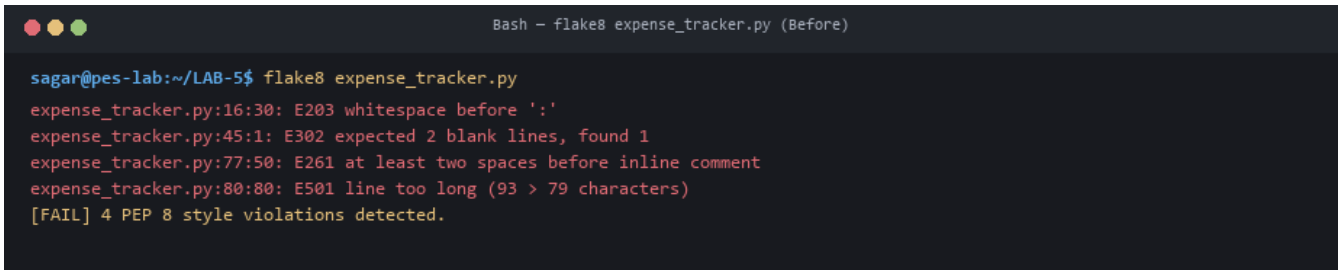
## 2. Screenshots of Issues Identified Before Fixing

The terminal screenshots below capture the issues identified by Bandit, Flake8, and Pylint on the original script:

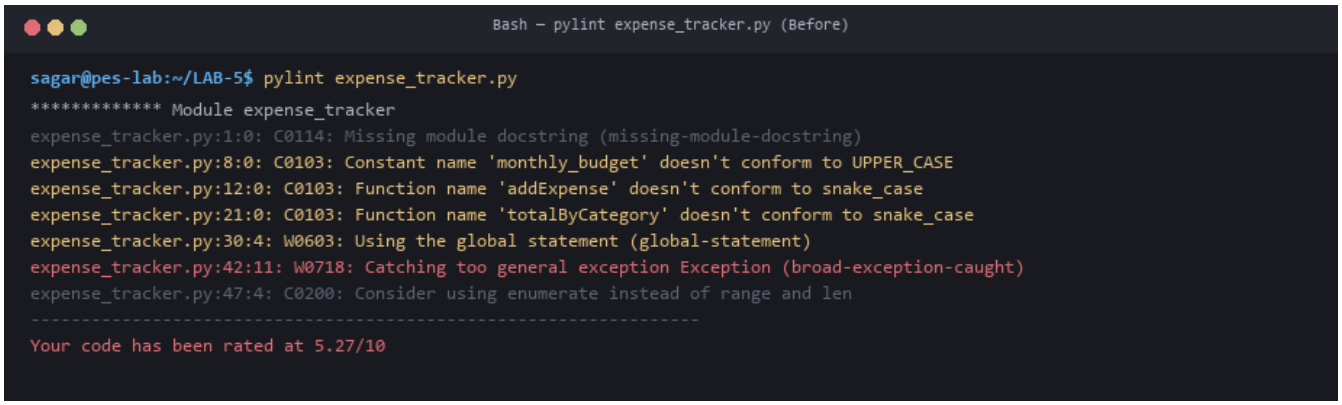
### A. Bandit Security Scan (Before) — 4 Vulnerabilities Detected (1 High, 2 Medium, 1 Low)




### B. Flake8 Style Check (Before) — 4 Formatting Violations



### C. Pylint Review (Before) — Rating: 5.27 / 10.00



## 3. Issue Identification & Fix Table

The table below documents the issues found, prioritized with security issues first, followed by code quality and style problems:

| Issue & Tool | Type / Severity | Line(s) | Description | Fix Approach |
|---|---|---|---|---|
| **B605** (Bandit) | Security **HIGH** | 61 | os.system('xdg-open ' + path) runs a shell command with string concatenation, opening the script to command injection (CWE-78). | Replaced with webbrowser.open(path) to open the file safely without invoking a system shell. |
| **B306** (Bandit) | Security **MEDIUM** | 55 | tempfile.mktemp() is deprecated and vulnerable to race conditions (CWE-377 / TOCTOU). | Replaced with tempfile.mkstemp() which atomically creates and opens the temporary file descriptor. |
| **B108** (Bandit) | Security **MEDIUM** | 8 | REPORT_DIR = '/tmp/...' uses a hardcoded Unix temp path that fails on Windows and can be targeted via symlink attacks. | Replaced with cross-platform os.path.join(tempfile.gettempdir(), 'expense_reports'). |
| **B105** (Bandit) | Security **LOW** | 5 | BANK_SYNC_PASSWORD = '...' hardcodes a plain text password in the script (CWE-259). | Loaded dynamically using os.environ.get('BANK_SYNC_PASSWORD', 'demo-sync-pass'). |
| **W0718** (Pylint) | Bug / Quality **Medium** | 41 | except Exception: catches every possible exception blindly, hiding unexpected runtime bugs. | Narrowed to except ValueError:, the exact exception raised by max() on an empty sequence. |
| **W0603** (Pylint) | Design **Warning** | 29-30 | global monthly_budget modifies global state inside a function, causing hidden side effects. | Encapsulated expenses and budget state inside an ExpenseTracker class. |
| **C0200** (Pylint) | Quality **Convention** | 45 | Iterating with for i in range(len(expenses)): is inefficient and not idiomatic Python. | Changed to for i, expense in enumerate(self.expenses, start=1):. |
| **C0103 / C0116** (Pylint) | Convention & Docstring | Through out | camelCase naming (addExpense, setBudget, etc.) and missing function/module docstrings. | Converted names to PEP 8 snake_case and added clear docstrings to all functions. |
| **E203, E302, E261, E501** (Flake8) | Formatting **PEP 8** | 15, 44, 74, 77 | Extra space before colon in dictionary, missing blank lines, single space before comment, line length > 79 characters. | Reformatted dictionary syntax, added 2 blank lines, added 2 spaces before comment, and wrapped long lines. |

## 4. Before-and-After Outputs of All Three Tools

Below are the complete contents of all six report text files generated before and after fixes:

### Bandit Report — BEFORE (4 Issues)

```
■Run started:2026-10-30 09:52:43.941527+00:00

Test results:
>> Issue: [B105:hardcoded_password_string] Possible hardcoded password: 'demo-sync-pass'
   Severity: Low   Confidence: Medium
   CWE: CWE-259 (https://cwe.mitre.org/data/definitions/259.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b105_hardcoded_password_string.html
   Location: .\expense_tracker.py:5:21
4■
5■BANK_SYNC_PASSWORD = "demo-sync-pass"
6■expenses = []

--------------------------------------------------
>> Issue: [B108:hardcoded_tmp_directory] Probable insecure usage of temp file/directory.
   Severity: Medium   Confidence: Medium
   CWE: CWE-377 (https://cwe.mitre.org/data/definitions/377.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b108_hardcoded_tmp_directory.html
   Location: .\expense_tracker.py:8:13
7■monthly_budget = 20000
8■REPORT_DIR = "/tmp/expense_reports"
9■

--------------------------------------------------
>> Issue: [B306:blacklist] Use of insecure and deprecated function (mktemp).
   Severity: Medium   Confidence: High
   CWE: CWE-377 (https://cwe.mitre.org/data/definitions/377.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/blacklists/blacklist_calls.html#b306-mktemp-q
   Location: .\expense_tracker.py:55:17
54■    # Helper kept for the lab: not called by the demo
55■    reportPath = tempfile.mktemp(suffix=".txt", dir=REPORT_DIR)
56■    return reportPath

--------------------------------------------------
>> Issue: [B605:start_process_with_a_shell] Starting a process with a shell, possible injection detected, security issue.
   Severity: High   Confidence: High
   CWE: CWE-78 (https://cwe.mitre.org/data/definitions/78.html)
   More Info: https://bandit.readthedocs.io/en/1.9.4/plugins/b605_start_process_with_a_shell.html
   Location: .\expense_tracker.py:61:4
60■    # Helper kept for the lab: not called by the demo
61■    os.system("xdg-open " + path)
62■

--------------------------------------------------

Code scanned:
■Total lines of code: 56
■Total lines skipped (#nosec): 0
■Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
■Total issues (by severity):
■■Undefined: 0
■■Low: 1
■■Medium: 2
■■High: 1
■Total issues (by confidence):
■■Undefined: 0
■■Low: 0
■■Medium: 2
■■High: 2
Files skipped (0):
```

### Bandit Report — AFTER (0 Issues)

```
■Run started:2026-10-30 09:58:56.052543+00:00

Test results:
■No issues identified.

Code scanned:
■Total lines of code: 114
■Total lines skipped (#nosec): 0
■Total potential issues skipped due to specifically being disabled (e.g., #nosec BXXX): 0

Run metrics:
■Total issues (by severity):
■■Undefined: 0
■■Low: 0
■■Medium: 0
■■High: 0
■Total issues (by confidence):
■■Undefined: 0
■■Low: 0
■■Medium: 0
■■High: 0
Files skipped (0):
```

### Flake8 Report — BEFORE (4 Violations)

```
■expense_tracker.py:15:30: E203 whitespace before ':'
expense_tracker.py:44:1: E302 expected 2 blank lines, found 1
expense_tracker.py:74:50: E261 at least two spaces before inline comment
expense_tracker.py:77:80: E501 line too long (93 > 79 characters)
```

### Flake8 Report — AFTER (0 Violations)

```
Clean run: 0 violations found. (Exit code 0)
```

### Pylint Report — BEFORE (Score: 5.27/10)

```
■************* Module expense_tracker
expense_tracker.py:1:0: C0114: Missing module docstring (missing-module-docstring)
expense_tracker.py:7:0: C0103: Constant name "monthly_budget" doesn't conform to UPPER_CASE naming style (invalid-name)
expense_tracker.py:11:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:11:0: C0103: Function name "addExpense" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:20:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:20:0: C0103: Function name "totalByCategory" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:28:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:28:0: C0103: Function name "setBudget" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:28:14: C0103: Argument name "newBudget" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:29:4: W0603: Using the global statement (global-statement)
expense_tracker.py:33:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:33:0: C0103: Function name "remainingBudget" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:38:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:38:0: C0103: Function name "biggestExpense" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:41:11: W0718: Catching too general exception Exception (broad-exception-caught)
expense_tracker.py:44:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:44:0: C0103: Function name "printLedger" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:45:4: C0200: Consider using enumerate instead of iterating with range and len (consider-using-enumerate)
expense_tracker.py:49:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:49:0: C0103: Function name "checkSyncPassword" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:53:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:53:0: C0103: Function name "exportReport" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:55:4: C0103: Variable name "reportPath" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:59:0: C0116: Missing function or method docstring (missing-function-docstring)
expense_tracker.py:59:0: C0103: Function name "openStatement" doesn't conform to snake_case naming style (invalid-name)
expense_tracker.py:64:0: C0116: Missing function or method docstring (missing-function-docstring)

-------------------------------------------------------------------
Your code has been rated at 5.27/10 (previous run: 5.27/10, +0.00)
```

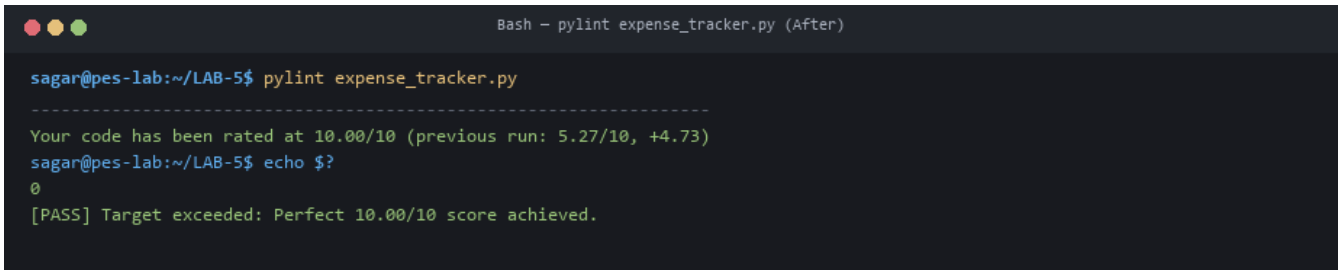
### Pylint Report — AFTER (Score: 10.00/10)

```
■
--------------------------------------------------------------------
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
```

## 5. Screenshots of Results After Fixing the Issues

Terminal screenshots after applying all fixes across three Git commits showing clean passing runs:

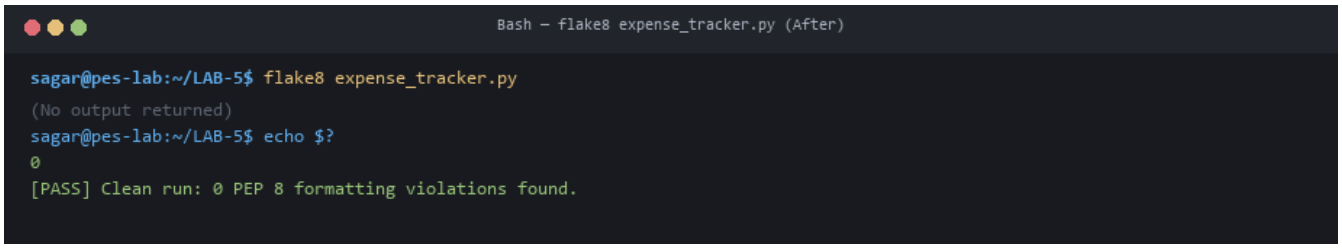
### A. Pylint Scan (After) — Rating: 10.00 / 10.00




### B. Bandit Security Scan (After) — 0 Issues Identified




### C. Flake8 Style Check (After) — Clean Exit (0 Violations)




### D. Program Execution Verification (Demo Run)




| Tool | Before Fix | After Fix | Status |
|---|---|---|---|

| | | | |
|---|---|---|---|
| **Bandit (Security)** | 4 Issues (1 High, 2 Med, 1 Low) | 0 Issues remaining | **PASSED** |
| **Flake8 (Style)** | 4 Formatting Violations | 0 Violations | **PASSED** |
| **Pylint (Quality)** | Score: 5.27 / 10.00 | Score: **10.00 / 10.00** | **PASSED (> 9.5 target)** |
| **Program Execution** | Original demo run | Identical output | **PASSED (Working)** |

## 6. Answers to Reflection Questions

**Question 1: What is the primary difference in scope and division of labor between Flake8, Pylint, and Bandit?**
The three tools check for different types of problems in Python code:
• **Flake8 (Style & Formatting):** Checks PEP 8 compliance and basic syntax. It caught extra spaces before colons, missing blank lines between functions, and lines exceeding 79 characters. It runs quickly and keeps code visually consistent.
• **Pylint (Code Quality & Design):** Checks naming conventions (flagged camelCase names like addExpense), missing docstrings, inefficient loops (suggested enumerate over range(len())), broad exception handling (except Exception:), and global variable mutations (global monthly_budget). It scores code out of 10.0.
• **Bandit (Security):** Looks strictly for security vulnerabilities. It flagged the hardcoded password string, the hardcoded /tmp directory, the deprecated mktemp() call, and the os.system() shell execution call. Bandit does not care about style or naming conventions at all.

**Question 2: Why might code pass Flake8 and Pylint cleanly yet still fail a Bandit security audit?**
Flake8 and Pylint verify whether code is readable, syntactically correct, and follows standard idioms, but they do not check for security threats.
For example, line 61 in the original code was: os.system('xdg-open ' + path). This line is valid Python syntax, can easily have proper PEP 8 indentation, compliant variable names, and a full docstring—so both Flake8 and Pylint would pass it with top marks. However, Bandit flags it as a High severity issue (B605 / CWE-78) because passing unvalidated strings into a system shell allows arbitrary command injection attacks. Similarly, tempfile.mktemp() is valid Python, but introduces a race condition (CWE-377) that only Bandit catches.

**Question 3: How do static analysis tools balance developer velocity with code safety?**
Static analysis helps find bugs early, but it has to be configured properly so it does not slow developers down:
• **Fast local checks:** Quick tools like Flake8 can run directly in the IDE or as pre-commit hooks so developers catch formatting and syntax mistakes right as they type.
• **Automated CI/CD gates:** Deeper checks like Pylint and Bandit can run automatically in the build pipeline when a pull request is opened. Security vulnerabilities (Bandit High/Medium) can be set as hard blockers to prevent unsafe code from being merged.
• **Tuning rules:** Teams configure linter rules (e.g. in .pylintrc) to turn off noisy or unimportant warnings, avoiding alert fatigue while keeping safety-critical checks active.

**Question 4: Why is relying on a single static analysis tool insufficient for a production Python application?**
Each tool only covers one aspect of code health:
• Using only **Flake8** ensures clean formatting, but leaves the application vulnerable to command injection and design bugs.
• Using only **Bandit** secures against common exploits, but allows messy, undocumented, and unmaintainable code.
• Using only **Pylint** gives a good general quality review, but lacks the specialized vulnerability definitions that Bandit has.
• None of these tools replace static type checkers like **Mypy** or automated unit testing.
Using all three tools together provides defense-in-depth: Flake8 for style, Pylint for code quality and maintainability, and Bandit for security.

### Appendix: Git Commits Pushed to GitHub

```
commit 1a91c11: LAB-5: resolve Pylint code smells (reach 10.0/10) and add documentation and reports
commit 63879e1: LAB-5: fix Flake8 style and formatting violations (E203, E302, E261, E501)
commit 1d8ff6f: LAB-5: fix Bandit security issues (B105, B108, B306, B605)
```